

Vehicle Portfolio Management System

Project Overview

This project is a comprehensive full-stack web application designed to help fleet owners manage their vehicles, drivers, trips, fuel (diesel), servicing, and financial analytics. It provides role-based access for Owners, Drivers, and Admins.

How the Project Works

The system follows a classic client-server architecture:

- Frontend (Vanilla HTML/CSS/JS):** Provides the user interface. It uses Bootstrap 5 for responsive styling. The frontend communicates with the backend via RESTful API calls using the native `fetch` API. It uses JWT (JSON Web Tokens) for authentication.
- Backend (Node.js + Express):** Serves as the API layer. It handles business logic, database queries, and file uploads. It exposes various endpoints (e.g., `/api/vehicles`, `/api/trips`) protected by authentication middleware.
- Database (MySQL):** Stores all persistent data including users, vehicles, trips, expenses, and service records.
- File Storage:** Uploaded files (like diesel bills or vehicle photos) are stored locally in the backend `uploads/` directory, and their paths are saved in the database.

1. Backend Code Structure & Explanations

`backend/server.js`

Purpose: The main entry point for the backend API.

Code Explanation:

- Initializes the Express application.
- Configures middleware: `cors` for cross-origin requests, `express.json` and `express.urlencoded` for parsing request bodies.
- Serves static files from the `uploads` directory so the frontend can access uploaded images.
- Mounts all the route modules (e.g., `app.use('/api/auth', authRoutes)`).
- Starts the HTTP server on the port defined in `.env`.

`backend/config/db.js`

Purpose: Database connection configuration.

Code Explanation:

- Uses the `mysql2/promise` library to create a connection pool to the MySQL database.
- Reads connection details (`host`, `user`, `password`, `db name`) from environment variables using `dotenv`.
- Exports the configured pool so other modules can execute queries.

`backend/middleware/auth.js`

Purpose: Protects API routes from unauthorized access.

Code Explanation:

- Exports a `verifyToken` function.
- Reads the `Authorization` header from incoming requests.
- Uses `jsonwebtoken (jwt.verify)` to validate the token.
- If valid, attaches the decoded user payload to `req.user` and calls `next()`. If invalid, returns a 401 or 403 error.

Backend Routes (`backend/routes/`)

These files define the API endpoints and contain the core business logic.

- `auth.js`:** Handles `/api/auth/register` (hashes passwords with `bcryptjs` and inserts users) and `/api/auth/login` (verifies passwords and signs JWTs).
- `vehicles.js`:** Handles CRUD operations for vehicles. The `POST` route validates input, inserts a new vehicle record, and links it to the owner. The `GET` route retrieves vehicles specific to the logged-in user's role.
- `trips.js`:** Manages trip data. Code calculates total toll fees, revenue, and links trips to specific vehicles.
- `diesel.js`:** Manages fuel records. Handles creating diesel entries and calculating costs per vehicle/trip.
- `service.js`:** Manages service records and maintenance alerts. The code compares `next_service_date` with the current date to trigger alerts for upcoming servicing.
- `uploads.js`:** Uses the `multer` library to handle multipart/form-data. It defines storage configurations (saving files to specific folders based on `doc_type`) and saves file metadata to the `uploads` table.
- `financials.js / analytics.js`:** Contains complex SQL aggregation queries (using `SUM()`, `GROUP BY`) to calculate total revenue, total expenses, profit, and monthly trends for the dashboard charts.
- `ocr.js`:** Uses the `tesseract.js` library to perform Optical Character Recognition on uploaded documents to auto-extract text (like vehicle numbers).

`backend/database/setup.js`

Purpose: Database schema initialization.

Code Explanation:

- Contains the SQL `CREATE TABLE` statements for the entire database (users, vehicles, trips, diesel_records, etc.).
- Defines foreign key relationships and cascades.
- Run manually (`node database/setup.js`) to set up a fresh database environment.

2. Frontend Code Structure & Explanations

Core Layout and Shared Logic

- `index.html` & `js/auth.js`:** The login/registration page. The JS code handles form submission, sends credentials to the backend, and stores the received JWT token in `localStorage`.
- `js/api.js`:** A central utility file containing common functions like `getToken()`, `getCurrentUser()`, `logout()`, and `showAlert()`. It prevents code duplication.

across different pages.

Dashboard & Analytics

- **dashboard.html & js/dashboard.js**: The main landing page after login. The JS fetches the user's vehicles and displays them as cards. It parses the `photo_url` JSON to display the primary vehicle image. It also fetches service alerts to display warnings.
- **analytics.html & js/analytics.js**: Displays financial graphs. The JS fetches aggregated data from the backend and uses the `Chart.js` library to render revenue vs. expense bar charts and monthly trend line charts.

Vehicle Management

- **add-vehicle.html & js/vehicle.js**: Form to register a new vehicle. The JS handles multiple photo uploads, converts the selected files to `FormData`, uploads them to `/api/uploads/photo`, and then sends the vehicle JSON payload to `/api/vehicles`.
- **vehicle-detail.html**: Shows comprehensive details for a single vehicle. The JS (`vehicle.js`) uses URL parameters to fetch specific vehicle data, renders a Bootstrap carousel for multiple photos, and loads associated trips and service records into tabs.

Operations (Trips, Diesel, Service, Compliance)

- **trips.html & js/trips.js**: Interface to log journeys. The JS dynamically populates the vehicle dropdown, submits trip data, and renders a table of past trips.
- **diesel.html & js/diesel.js**: Interface for fuel logs. Users can upload a photo of the fuel bill. The JS handles the multipart upload and links it to a specific trip.
- **service.html & js/service.js**: Maintenance tracking. Displays past service records and upcoming deadlines.
- **compliance.html & js/compliance.js**: Tracks document renewals (Insurance, RC, Permits). The JS calculates the days remaining until expiry and color-codes the table rows (Red for expired, Yellow for soon).

Driver Portal

- **driver-register.html**: A specialized registration form for drivers to build their profile (license details, experience).
- **hire-drivers.html**: For owners to browse available drivers and send job requests.
- **driver-dashboard.html**: The specific view for logged-in drivers, showing the vehicle assigned to them and their assigned trips.

Summary of Execution Flow

1. **Authentication**: User logs in -> Token stored in `localStorage` -> Included as `Bearer <token>` in subsequent API requests.
2. **Data Retrieval**: Page loads -> JS `DOMContentLoaded` fires -> `fetch` calls backend API -> Backend verifies token -> Queries DB -> Returns JSON -> JS dynamically manipulates DOM to render data.
3. **File Uploads**: User selects files -> JS creates `FormData` -> `fetch` posts to `/api/uploads` -> Backend `multer` saves file to disk -> Returns file URL -> JS attaches URL to main entity payload (e.g., Vehicle or Trip).